

Reviewer Pack — Minimal Ehrhart Polynomial Complexity and Communication Matr...

Entry 25c8b70e2609 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 11:28:08 UTC

Minimal Ehrhart Polynomial Complexity and Communication Matrix Rank

Entry ID: 25c8b70e2609 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 11:28:08 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Ehrhart Theory) **Field B** (complexity object): Communication Complexity

Statement:

For any given Boolean function f , the communication matrix rank of its representation in a complete binary tree is linearly correlated with the degree of the minimal Ehrhart polynomial of the corresponding zonotope, such that the communication matrix rank $R(f) = \Theta(\deg(\text{EhrhartPolynomial}(\text{Zonotope}(f))))$, where $\deg$ denotes the degree of a polynomial.

Rationale (proposer's reasoning):

Ehrhart Theory studies the counting of lattice points in integer polyhedra and has recently found connections to computational complexity. By exploring the connection between Ehrhart polynomials and communication complexity, this conjecture aims to reveal new insights into both fields. The minimal Ehrhart polynomial captures the complexity of counting lattice points, which could be related to the complexity of distributing information in communication complexity.

Taxonomy category: EHRHART_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2bc23aa0638d8ba2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 30 independent random seeds, the communication matrix rank $R(f)$ of a Boolean function f 's complete binary tree representation is within a constant factor of $\Theta(\deg(\text{EhrhartPolynomial}(\text{Zonotope}(f))))$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Ehrhart polynomial" AND "communication complexity" AND "degree correlation" - "zonotope" IN title AND communication matrix rank AND Ehrhart theory" - "Boolean function" AND complete binary tree representation AND minimal Ehrhart polynomial degree

Top relevant hits considered: - [<http://arxiv.org/abs/1910.12458v3>] On the Degree of Boolean Functions as Polynomials over $\mathbb{Z}_m$ - [<http://arxiv.org/abs/1307.1157v1>] Representing Boolean Functions Using Polynomials: More Can Offer Less - [<http://arxiv.org/abs/1803.09947v3>] Periodic Fourier representation of Boolean functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=241.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def zonotope_vertices(f):
        n = len(f)
        vertices = []
        for i in range(2**n):
            vertex = [f[i >> j & 1] * (i >> (j + 1) & 1) for j in range(n)]
            vertices.append(vertex)
        return vertices

    def min_ehrhart_polynomial(vertices):
        n = len(vertices[0])
        A = [[sum(v[j] for v in vertices if v[i] == 1) - sum(v[j] for v in vertices if v[i] == 0)
              for j in range(n)] for i in range(n)]
        det = determinant(A)
        return det

    def determinant(matrix):
        n = len(matrix)
        if n == 1:
            return matrix[0][0]
        det = 0
        for i in range(n):
            sign = (-1)**(i+n)
            minor = []
            for r in range(1, n):
                for c in range(1, n):
                    if c != i:
                        minor.append(matrix[r][c])
            det += sign * matrix[0][i] * determinant(minor)
        return det
```

```

n = len(matrix)
if n == 1:
    return matrix[0][0]
det = 0
for j in range(n):
    submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
    det += (-1)**j * matrix[0][j] * determinant(submatrix)
return det

def communication_matrix_rank(f):
    n = len(f)
    tree = [[] for _ in range(2*n-1)]
    queue = [0]
    while queue:
        node = queue.pop(0)
        if node < n:
            left_child = 2*node + 1
            right_child = 2*node + 2
            tree[node].append(left_child)
            tree[node].append(right_child)
            queue.append(left_child)
            queue.append(right_child)
    rank = 0
    visited = [False] * (2*n-1)
    for i in range(n):
        if not visited[i]:
            stack = [i]
            while stack:
                node = stack.pop()
                if not visited[node]:
                    visited[node] = True
                    for neighbor in tree[node]:
                        stack.append(neighbor)
            rank += 1
    return rank

n = random.choice([5, 10, 15, 20, 30, 40])
f = generate_boolean_function(n)
vertices = zonotope_vertices(f)
ehrhart_poly = min_ehrhart_polynomial(vertices)
comm_rank = communication_matrix_rank(f)

if ehrhart_poly == 0:
    return {
        "metric_name": "communication_matrix_rank",
        "metric_value": comm_rank,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

degree = len(bin(ehrhart_poly)) - 2
if comm_rank == 0:
    return {
        "metric_name": "communication_matrix_rank",

```

```

        "metric_value": comm_rank,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

ratio = abs(comm_rank / degree)
return {
    "metric_name": "communication_matrix_rank",
    "metric_value": comm_rank,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": ratio <= 2 and ratio >= 0.5,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.randint(2, 1000003) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results]
    conjecture_holds = all(r["conjecture_holds"] for r in results)
    counterexample = next((r["counterexample"] for r in results if not r["conjecture_holds"]), "")

    mean_value = sum(metric_values) / len(metric_values)
    std_value = math.sqrt(sum((x - mean_value)**2 for x in metric_values) / len(metric_values))

    if conjecture_holds:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction=1.0")
    elif counterexample:
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seeds[results.index(counterexample)]}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the expected time frame. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	46122
2	preregistration	ollama_remote	glm4:latest	0	0	9953
3	novelty	ollama_remote	glm4:latest	0	0	14892
4	novelty	ollama_remote	glm4:latest	0	0	10268
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18063
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	235361
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	32530
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18910
9	judge	ollama_remote	glm4:latest	0	0	70193

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 456292 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/25c8b70e2609.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/25c8b70e2609.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/25c8b70e2609.tar.gz (if generated)